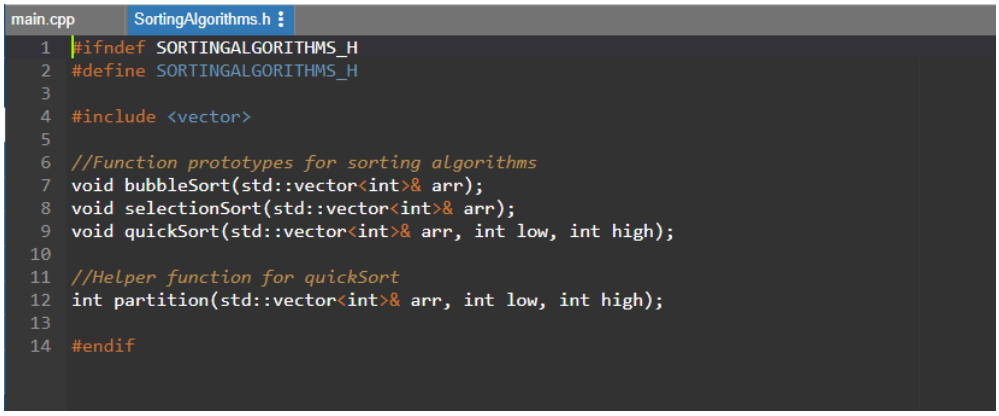
ACTIVITY NO. 7	
SORTING ALGORITHMS: BUBBLE, SELECTION, AND INSERTION SORT	
Course Code: CPE010	**Program:** Computer Engineering
Course Title: Data Structures and Algorithms	**Date Performed:** October 16, 2024
Section: CPE21S4	**Date Submitted:** October 18, 2024
Name(s): Gob, Mark Jeonel Kenn E.	**Instructor:** Ma'am Maria Rizette Sayo

6. Output

Code + Console Screenshot



Observations	I observe that the C++ program generates a random array of integers, prints the array, and then exits. The program uses the ***srand*** function to seed the random number generator and the ***rand*** function to generate random numbers. The array is then printed using a ***for*** loop.

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	main.cpp ```cpp #include <iostream> #include <utility> template <typename T> void bubbleSort(T arr[], size_t arrSize) { //Step 1: For i = 0 to N-1 repeat Step 2 for (size_t i = 0; i < arrSize; i++) { //Step 2: For j = i + 1 to N - 1 repeat for (size_t j = i + 1; j < arrSize; j++) { //Step 3: if arr[j] > arr[i] if (arr[j] > arr[i]) { // Swap arr[j] and arr[i] std::swap(arr[j], arr[i]); } } } //Step 4: Exit } //Function to print array template <typename T> void printArray(T arr[], size_t arrSize) { for (size_t i = 0; i < arrSize; i++) { std::cout << arr[i] << " "; } std::cout << std::endl; } int main() { //Array of values for sorting int arr[] = {94, 78, 25, 46, 83, 33, 48, 17, 29, 88, 56, 5, 96, 21, 65, 57, 96, 87, 73, 50, 8, 15, 58, 9, 67, 84, 52, 44, 15, 7, 9, 62, 85, 35, 8, 68, 20, 8, 86, 1, 97, 42, 7, 45, 64, 72, 2, 12, 11, 27, 15, 20, 42, 73, 81, 61, 9, 33, 57, 24, 40, 66, 86, 26, 53, 46, 94, 25, 55, 32, 27, 52, 27, 34, 97, 43, 6, 51, 55, 17, 30, 22, 89, 24, 95, 22, 37, 4, 55, 94, 81, 48, 60, 67, 26, 66, 66, 20, 91, 21}; size_t arrSize = sizeof(arr) / sizeof(arr[0]); std::cout << "Original array: "; printArray(arr, arrSize); bubbleSort(arr, arrSize); std::cout << "Sorted array: "; printArray(arr, arrSize); return 0; } ``` input ``` Original array: 94 78 25 46 83 33 48 17 29 88 56 5 96 21 65 57 96 87 73 50 8 15 58 9 67 84 52 44 15 7 9 62 85 35 8 68 20 8 86 1 97 42 7 45 64 72 2 12 11 27 15 20 42 73 81 61 9 33 57 24 40 66 86 26 53 46 94 25 55 32 27 52 27 34 97 43 6 51 55 17 30 22 89 24 95 22 37 4 55 94 81 48 60 67 26 66 66 20 91 21 Sorted array: 97 97 96 96 95 94 94 94 91 89 88 87 86 86 85 84 83 81 81 78 73 73 72 68 67 67 66 66 66 65 64 62 61 60 58 57 57 56 55 55 55 53 52 52 51 50 48 48 46 46 45 44 43 42 42 40 37 35 34 33 33 32 30 29 27 27 27 26 26 25 25 24 24 22 22 21 21 20 20 20 17 17 15 15 15 12 11 9 9 9 8 8 8 7 7 6 5 4 2 1 ...Program finished with exit code 0 Press ENTER to exit console. ```
Observations	I observe that the program implements a bubble sort algorithm to sort the retrieved array of integers from the previous program. The program first prints the original array, then performs the bubble sort algorithm, and finally prints the sorted array. The bubble sort algorithm works by repeatedly swapping adjacent elements if they are in the wrong order until the array is fully sorted.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	main.cpp ```cpp #include <iostream> #include <utility> // For std::swap //Routine to find the position of the smallest element template <typename T> int Routine_Smallest(T A[], int K, const int arrSize) { int position = K; //Initialize position to K T smallestElem = A[K]; //Initialize smallestElem to A[K] //Step 3: For J = K + 1 to N - 1, repeat for (int J = K + 1; J < arrSize; J++) { if (A[J] < smallestElem) { //If a smaller element is found smallestElem = A[J]; //Update smallestElem position = J; //Update position } } //Step 4: Return position of the smallest element return position; } //Selection Sort function template <typename T> void selectionSort(T arr[], const int N) { int POS, temp; //Step 1: Repeat Steps 2 and 3 for K = 0 to N - 1 for (int i = 0; i < N; i++) { //Step 2: Call routine smallest(A, K, N) POS = Routine_Smallest(arr, i, N); //Step 3: Swap A[K] with A[POS] std::swap(arr[i], arr[POS]); } ``` input

```
    //Step 4: Exit
}

//Function to print the array
template <typename T>
void printArray(T arr[], const int arrSize) {
    for (int i = 0; i < arrSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

int main() {
    //New array of values for sorting
    int arr[] = {94, 78, 25, 46, 83, 33, 48, 17, 29, 88, 56, 5, 96, 21, 65,
                 57, 96, 87, 73, 50, 8, 15, 58, 9, 67, 84, 52, 44, 15,
                 7, 9, 62, 85, 35, 8, 68, 20, 8, 86, 1, 97, 42, 7,
                 45, 64, 72, 2, 12, 11, 27, 15, 20, 42, 73, 81, 61,
                 9, 33, 57, 24, 40, 66, 86, 26, 53, 46, 94, 25, 55,
                 32, 27, 52, 27, 34, 97, 43, 6, 51, 55, 17, 30, 22,
                 89, 24, 95, 22, 37, 4, 55, 94, 81, 48, 60, 67, 26,
                 66, 66, 20, 91, 21};

    const int arrSize = sizeof(arr) / sizeof(arr[0]);

    std::cout << "Original array: ";
    printArray(arr, arrSize);

    selectionSort(arr, arrSize);

    std::cout << "Sorted array: ";
    printArray(arr, arrSize);

    return 0;
```

```
Original array: 94 78 25 46 83 33 48 17 29 88 56 5 96 21 65 57 96 87 73 50 8 15 58 9 67 84 52 44 15 7 9 62 85 35 8 68 20 8 86 1 97 42 7 45 64 72 2 12 11 27 15 20 42 73 81 61 9 33 57 24 40
66 86 26 53 46 94 25 55 32 27 52 27 34 97 43 6 51 55 17 30 22 89 24 95 22 37 4 55 94 81 48 60 67 26 66 66 20 91 21
Sorted array: 1 2 4 5 6 7 7 8 8 8 9 9 9 11 12 15 15 15 17 17 20 20 20 21 21 22 22 24 24 25 25 26 26 27 27 27 29 30 32 33 33 34 35 37 40 42 42 43 44 45 46 46 48 48 50 51 52 52 53 55 55 55 5
6 57 57 58 60 61 62 64 65 66 66 66 67 67 68 72 73 73 78 81 81 83 84 85 86 86 87 88 89 91 94 94 94 95 96 96 97 97


...Program finished with exit code 0
Press ENTER to exit console.
```

Observations	I observe that the program implements the selection sort algorithm to sort the array of integers acquired earlier from the random generator program in 7-1. The program first prints the original array, then performs the selection sort algorithm, and finally prints the sorted array. The selection sort algorithm works by repeatedly finding the minimum element in the unsorted part of the array and swapping it with the first element of the unsorted part.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot

main.cpp

```
#include <iostream>

//Insertion Sort function
template <typename T>
void insertionSort(T arr[], const int N) {
    int K = 1; //Start from the second element
    T temp; //Temporary variable for the current element

    //Step 1: Repeat Steps 2 to 5 for K = 1 to N-1
    while (K < N) {
        //Step 2: Set temp = A[K]
        temp = arr[K];

        //Step 3: Set J = K - 1
        int J = K - 1;

        //Step 4: Repeat while J >= 0 and temp < A[J]
        while (J >= 0 && temp < arr[J]) {
            //Set A[J + 1] = A[J]
            arr[J + 1] = arr[J];
            //Set J = J - 1
            J--;
        }
        //Step 5: Set A[J + 1] = temp
        arr[J + 1] = temp;

        //Increment K for the next iteration
        K++;
    }
    //Step 6: Exit
}
```

```
//Function to print the array
template <typename T>
void printArray(T arr[], const int arrSize) {
    for (int i = 0; i < arrSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

int main() {
    //Example usage
    int arr[] = {94, 78, 25, 46, 83, 33, 48, 17, 29, 88, 56, 5, 96, 21, 65,
                 57, 96, 87, 73, 50, 8, 15, 58, 9, 67, 84, 52, 44, 15,
                 7, 9, 62, 85, 35, 8, 68, 20, 8, 86, 1, 97, 42, 7,
                 45, 64, 72, 2, 12, 11, 27, 15, 20, 42, 73, 81, 61,
                 9, 33, 57, 24, 40, 66, 86, 26, 53, 46, 94, 25, 55,
                 32, 27, 52, 27, 34, 97, 43, 6, 51, 55, 17, 30, 22,
                 89, 24, 95, 22, 37, 4, 55, 94, 81, 48, 60, 67, 26,
                 66, 66, 20, 91, 21};

    const int arrSize = sizeof(arr) / sizeof(arr[0]);

    std::cout << "Original array: ";
    printArray(arr, arrSize);

    insertionSort(arr, arrSize);

    std::cout << "Sorted array: ";
    printArray(arr, arrSize);

    return 0;
```

input

```
Original array: 94 78 25 46 83 33 48 17 29 88 56 5 96 21 65 57 96 87 73 50 8 15 58 9 67 84 52 44 15 7 9 62 85 35 8 68 20 8 86 1 97 42 7 45 64 72 2 12 11 27 15 20 42
73 81 61 9 33 57 24 40 66 86 26 53 46 94 25 55 32 27 52 27 34 97 43 6 51 55 17 30 22 89 24 95 22 37 4 55 94 81 48 60 67 26 66 66 20 91 21
Sorted array: 1 2 4 5 6 7 7 8 8 8 9 9 9 11 12 15 15 15 17 17 20 20 20 21 21 22 22 24 24 25 25 26 26 27 27 27 29 30 32 33 33 34 35 37 40 42 42 43 44 45 46 46 48 48 50
 51 52 52 53 55 55 55 56 57 57 58 60 61 62 64 65 66 66 66 67 67 68 72 73 73 78 81 81 83 84 85 86 86 87 88 89 91 94 94 94 95 96 96 97 97


...Program finished with exit code 0
Press ENTER to exit console.
```

Observations	I observe that the program implements the insertion sort algorithm to sort the same array of integers generated earlier. The program first prints the original array, then performs the insertion sort algorithm, and finally prints the sorted array. The insertion sort algorithm works by iterating through the array, starting from the second element, and inserting each element into its correct position in the previously sorted subarray.

Table 7-4. Insertion Sort Algorithm

7. Supplementary Activity

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

List of Candidates

Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

NOTE: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.

Deliverables:
• Pseudocode of Algorithm
• Screenshot of Algorithm Code
• Output Testing

Pseudocode of Algorithm:

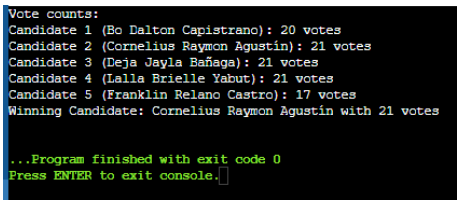
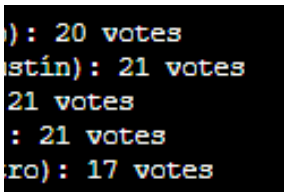
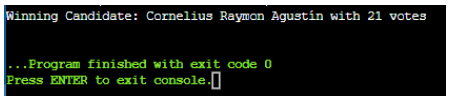
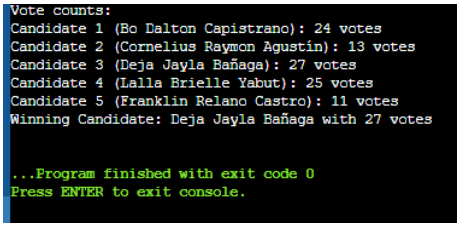
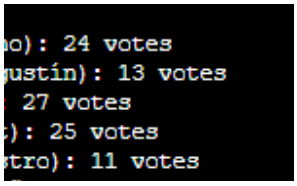
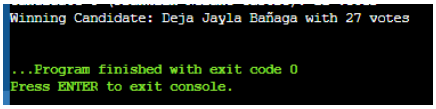
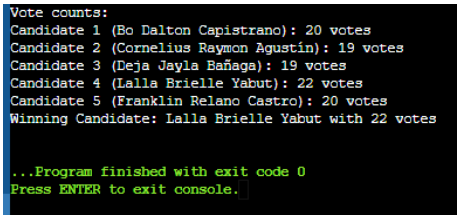
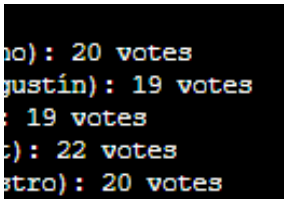
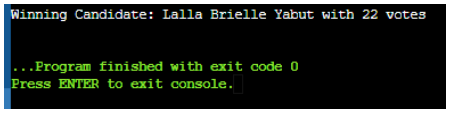
```
Unset
Start program

Initialize an array votes[100] to store random votes ranging from 1 to 5.
Define a list of candidates:
   - Candidate 1: "Bo Dalton Capistrano"
   - Candidate 2: "Cornelius Raymon Agustín"
   - Candidate 3: "Deja Jayla Bañaga"
   - Candidate 4: "Lalla Brielle Yabut"
   - Candidate 5: "Franklin Relano Castro"
Initialize an array voteCount[5] to zero for counting votes for each candidate.
```

```
For each index i from 0 to 99:
   a. Generate a random number between 1 and 5 and assign it to votes[i].

   For each index i from 0 to 99:
   a. If votes[i] is between 1 and 5:
      i. Increment voteCount[votes[i] - 1] by 1.

   Initialize maxVotes to 0 and winningCandidate to -1.

   For each index i from 0 to 4:
      a. If voteCount[i] is greater than maxVotes:
         i. Set maxVotes to voteCount[i].
         ii. Set winningCandidate to i.

   Print "Vote counts:"
      For each index i from 0 to 4:
         Print "Candidate (i + 1) (" + candidates[i] + "): " + voteCount[i] + " votes".

Print "Winning Candidate: " + candidates[winningCandidate] + " with " + maxVotes + " votes"

Stop Program
```

Screenshot of Algorithm Code:

```cpp
#include <iostream>
#include <cstdlib> //For rand() and srand()
#include <ctime>   //For time()

//Function to perform Bubble Sort
void bubbleSort(int arr[], const int N) {
    for (int i = 0; i < N - 1; i++) {
        for (int j = 0; j < N - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
            }
        }
    }
}

//Function to count votes and determine the winning candidate
void countVotes(int votes[], const int size, const std::string candidates[]) {
    int voteCount[5] = {0}; //Array to count votes for candidates 1 to 5

    //Count votes
    for (int i = 0; i < size; i++) {
        if (votes[i] >= 1 && votes[i] <= 5) {
            voteCount[votes[i] - 1]++;
        }
    }

    //Find the winning candidate
    int maxVotes = 0;
    int winningCandidate = -1;
    for (int i = 0; i < 5; i++) {
        if (voteCount[i] > maxVotes) {
            maxVotes = voteCount[i];
            winningCandidate = i; //Candidate numbers are 0 to 4
        }
    }

    //Print results
    std::cout << "Vote counts:\n";
    for (int i = 0; i < 5; i++) {
        std::cout << "Candidate " << (i + 1) << " (" << candidates[i] << "): " << voteCount[i] << " votes\n";
    }
    std::cout << "Winning Candidate: " << candidates[winningCandidate] << " with " << maxVotes << " votes\n";
}

int main() {
    const int size = 100;
    int votes[size];

    //Seed random number generator
    srand(static_cast<unsigned int>(time(0)));

    //Generate random votes between 1 and 5
    for (int i = 0; i < size; i++) {
        votes[i] = rand() % 5 + 1; //Random number between 1 and 5
    }

    //Candidate names
    std::string candidates[5] = {
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustín",
        "Deja Jayla Bañaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    //Sort the votes using Bubble Sort
    bubbleSort(votes, size);

    //Count votes and determine the winning candidate
    countVotes(votes, size, candidates);

    return 0;
}
```

Output Testing:

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
		
		
		

Question: Was your developed vote counting algorithm effective? Why or why not?

I chose to use Bubble Sort because it is a simple and intuitive sorting algorithm that is easy to implement, making it suitable for educational purposes such as the one we were tasked with in this HOA. Given the limited size of the dataset (100 elements) and the small range of values (1 to 5), Bubble Sort's time complexity is manageable. Additionally, Bubble Sort seems like a stable sorting algorithm for me, which preserves the relative order of equal elements, ensuring that the vote counting remains accurate and fair in that sense.

Furthermore, I believe that the developed vote counting algorithm was effective in accurately counting votes and determining the winning candidate. It successfully made use of a straightforward approach to tally votes and provide clear output, including the names of the candidates and their respective vote counts. However, while the algorithm works well for this specific scenario, I think that it may not be the most efficient choice for larger datasets or broader ranges of values, where more advanced sorting algorithms would be preferable for efficiency.

8. Conclusion

In this lesson, I gained a thorough understanding of basic sorting algorithms—Bubble, Selection, and Insertion Sort—and how to implement them using C++. By analyzing the procedures, I learned that while Bubble Sort and Selection Sort are simple and easy to follow, they are inefficient for large datasets, whereas Insertion Sort is more suitable for smaller or partially sorted data. The supplementary activity, which involved creating an algorithm to count votes using sorting techniques, was a practical application of these concepts, reinforcing the importance of choosing the right algorithm. I believe I performed well in writing the code, but I could improve by optimizing the time complexity of the algorithms and exploring more advanced techniques like Merge Sort which I encountered and had my interest piqued by when I was learning more about sorting techniques online. Going forward, I believe that I should focus on enhancing efficiency and deeper algorithmic analysis.

9. Assessment Rubric

Honor Pledge:

"I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own."